Take-Home Assignment: AI Document Review Experience

Context

AI agents are increasingly used to review documents and surface findings: risks, inconsistencies, missing clauses, and suggested changes. Presenting those findings well is a frontend challenge. Reviewers need to see each issue in context, understand why it matters, and work through everything efficiently.

This exercise is a self-contained scenario, designed so we can see how you approach that kind of problem. Your task is to design and build a document review interface where AI-generated findings live alongside the document and reviewers can efficiently work through them. Your submission is used only to evaluate your application.

Time budget: 4–6 hours. We respect your time. Scope accordingly and tell us what you prioritized and why. A focused, polished core beats a sprawling, half-finished feature list.

The Scenario

A legal operations reviewer at Northwind Analytics has received a vendor services agreement. An AI review agent has already analyzed it and produced about two dozen findings. The reviewer needs to:

1. Understand at a glance how "risky" the document is and where the problems are.
2. Work through the findings one by one, seeing each in the context of the surrounding text.
3. Decide what to do with each finding: accept it, dismiss it, or add their own note.
4. Know when they're done.

Materials

This email includes two attachments:

- **`sample-contract.json`**: a Master Services Agreement as structured content. The document has `sections`, each with an `id`, `number`, `heading`, and a list of `paragraphs`. Each paragraph has an `id`, `number`, and plain `text`.
- **`findings.json`**: sample output from an AI review agent. It contains agent metadata and a `findings` array. Each finding looks like this:

```
{
  "id": "f-08",
  "severity": "high",
  "category": "Liability",
  "title": "Liability cap limited to one month of fees",
  "explanation": "Vendor's total liability is capped at a single month of fees, which is far below market norms ...",
  "anchor": {
    "paragraphId": "p-9.2",
    "start": 81,
    "end": 199,
    "quote": "SHALL NOT EXCEED THE FEES PAID BY CUSTOMER TO VENDOR IN THE ONE (1) MONTH ..."
  },
  "suggestedEdit": "SHALL NOT EXCEED THE FEES PAID OR PAYABLE BY CUSTOMER TO VENDOR IN THE TWELVE (12) MONTHS ...",
  "confidence": 0.92
}
```


Field	Description
`severity`	`high`, `medium`, or `low`.
`confidence`	The agent's confidence in the finding, from 0 to 1.

Field	Description
`anchor`	The text the finding refers to. `start` and `end` are zero-based character offsets into the paragraph's `text` (end exclusive), and `quote` is the text the agent intended to reference. If a span continues into a later paragraph, `endParagraphId` is present and `end` refers to that paragraph. `anchor` is `null` for findings about the document as a whole, such as a missing clause.
`suggestedEdit`	Proposed replacement text for the anchored span, or `null` if the agent has no specific suggestion.

Treat the data as realistic, which means it isn't perfectly clean. How you load it is up to you: import it directly, or wrap it in a small mock service with simulated latency if you want to show loading states.

Core Requirements

- **Show findings in the context of the document.** A reviewer should be able to understand each finding alongside the text it refers to. How you present that relationship is up to you.
- **Show finding details:** severity, category, explanation, confidence, and suggested edit where present.
- **Make it easy to move between findings and the text they refer to,** in both directions.
- **Let reviewers act on findings:** accept, dismiss, or add a comment. Show review progress.
- **Help reviewers focus:** some way to filter or prioritize findings (by severity, category, or status, for example).
- **Handle edge cases gracefully,** including overlapping annotations and anchors that don't match cleanly.

Review state can live in memory or local storage. No backend is required.

Stretch Ideas (Optional)

Pick any that interest you, or none:

- Show suggested edits as an inline diff, with the option to apply them to the document.

- Keyboard-driven review (next/previous finding, accept/dismiss shortcuts).
- A summary or export of the completed review.
- Performance considerations for very long documents.
- Connecting a real LLM so reviewers can ask follow-up questions about a finding.

Technical Notes

- Set up your own project from scratch. Use any modern frontend framework. We use React and TypeScript, so those are welcome but not required.
- You may use component or UI libraries, but we're interested in the components *you* design, so avoid off-the-shelf solutions that do the core annotation work for you.
- No backend is required.
- Tests are welcome but not required for this task.

Deliverables

Reply to this email with:

1. A link to a **GitHub repository** with your code. Public is fine, or invite us to a private repo.
2. A **README** covering:
    - How to run it.
    - Your key design decisions, and the alternatives you considered.
    - What you prioritized, what you cut, and what you'd do with more time.
3. Optional: a **hosted demo** (Vercel, Netlify, and similar services work well) or a short (under 5 minutes) screen recording.

What We'll Look At

- **Modern user experience:** Does it feel like a contemporary product rather than a data table? Is it intuitive, fluid, and fast to work through?
- **Component design:** Are components well structured, reusable, and sensibly composed?
- **State and data handling:** How are review state, selection, and the document–finding relationship managed?
- **Robustness:** How are messy data, empty states, and loading states handled?
- **Visual polish and accessibility:** Attention to detail, keyboard support, and contrast.
- **Communication:** The clarity of your README and your reasoning about trade-offs.

In the follow-up interview, we'll walk through your solution together and discuss how it might evolve.

Questions? Reply to this email. We're happy to clarify anything, and if something is ambiguous, feel free to make a reasonable assumption and note it in your README.